

36-web-architecture

36. Web Architecture

Level: □ Beginner

Jam: 6 (4 sesi)

Prasyarat: — (lebih baik setelah 00-fundamentals)

Output: Architecture diagram + static site ter-deploy

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Paham cara kerja web: client, server, request-response cycle
- Bedain frontend vs backend dan tugas masing-masing
- Ngerti HTTP methods, status codes, dan headers
- Paham API dan REST, bisa tes endpoint pake Postman
- Tahu cara hosting static site dan bedain jenis hosting
- Baca URL, pake browser devtools, analisis network tab

Materi

Sesi	Topik	File
1	Cara Kerja Web — internet, client-server, URL, hosting	01-how-web-works.md
2	HTTP Basics — methods, status codes, headers, HTTPS	02-http-basics.md
3	Frontend vs Backend — API, REST, JSON, API testing	03-frontend-vs-backend.md
4	Hosting & Deployment — static/backend/database hosting, DNS, CI/CD	04-hosting-deploy.md

Output Akhir Modul

Architecture Diagram + Deployed Static Site —
gambar arsitektur web (client → DNS → CDN → server
→ database) pake Mermaid + static site (profile page)
ter-deploy di Vercel/GitHub Pages

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- “Explain this URL structure component by component”
- “Generate a Mermaid diagram of request-response cycle for a login flow”
- “What HTTP status code should I use for a resource not found?”
- “Refactor this API response to use proper JSON format”
- “Explain what happens when I type google.com in the browser”

01. Cara Kerja Web

Internet Basics

Internet = jaringan global komputer yang saling terhubung. Setiap perangkat punya alamat unik.

IP Address

- **IPv4:** 192.168.1.1 — 4 blok angka (32-bit), ~4.3 miliar alamat
- **IPv6:** 2001:db8::ff00:42:8329 — format hex (128-bit), alamat hampir tak terbatas

Cek IP sendiri
`curl ifconfig.me`

DNS (Domain Name System)

DNS = buku telepon internet. Manusia gampang inget google.com, komputer butuh IP 142.250.184.46.

Browser -> Cari DNS -> Dapet IP -> Connect ke server

Cek DNS lookup
`nslookup google.com`
`dig google.com +short`

TCP/IP

Protokol yang mengatur data dikirim via internet:

Layer	Fungsi	Contoh
Application	Data aplikasi	HTTP, FTP, SMTP
Transport	Pengiriman data	TCP, UDP
Internet	Routing & alamat	IP
Network Access	Hardware fisik	Ethernet, Wi-Fi

Client & Server

```
graph LR
    A[Client Browser] -->|Request| B[Web Server]
    B -->|Response| A
    B --> C[(Database)]
    style A fill:#e1f5fe,stroke:#01579b
    style B fill:#fff3e0,stroke:#e65100
    style C fill:#f3e5f5,stroke:#4a148c
```

Peran	Tugas	Contoh
Client	Minta data, render tampilan	Browser, mobile app, Postman
Server	Proses request, kirim response	Apache, Nginx, Node.js

Request-Response Cycle

```
sequenceDiagram
    participant User as User
    participant Browser as Browser
    participant Server as Web Server
    participant DB as Database

    User->>Browser: Ketik URL
    Browser->>Browser: DNS lookup
    Browser->>Server: HTTP Request (GET /)
    Server->>Server: Proses request
    Server->>DB: Query data
```

https://www.example.com:443/path/to/page?query=value&page=1#section

Protocol Domain Port Path Query Fragment

Cara buka: F12 atau Ctrl+Shift+I → tab **Network**

Kolom	Arti
Name	Nama file request
Status	HTTP status code
Type	Tipe resource (document, css, js, img)
Size	Ukuran response
Time	Waktu loading
Waterfall	Timeline request visual

Hosting Overview

Cara naro website biar bisa diakses dari internet.

Shared Hosting

- Satu server dipake banyak orang
- Murah, sumber daya terbatas
- Cocok: blog kecil, website statis

VPS (Virtual Private Server)

- Server virtual sendiri, resource dedicated
- Root access, bebas config
- Cocok: website menengah, API

Cloud Hosting

- Infrastruktur scalable (AWS, GCP, Azure)
- Bayar sesuai pemakaian
- Cocok: aplikasi besar, traffic fluktuatif

Serverless

- Jalani code tanpa ngurus server
- Scale otomatis, bayar per eksekusi
- Cocok: fungsi API, webhook, background job

```
graph LR
    subgraph "Hosting Types"
        A[Shared] -->|Small traffic| B[VPS]
        B -->|Medium traffic| C[Cloud]
        C -->|Spiky traffic| D[Serverless]
    end
    style A fill:#c8e6c9
    style B fill:#fff9c4
    style C fill:#ffccbc
    style D fill:#e1bee7
```

Rangkuman

Konsep	Inti
IP & DNS	IP itu alamat, DNS buku telepon
Client-Server	Client minta, server kasih
URL	Protocol://domain:port/path?query
DevTools Network	Liat semua request website
Hosting	Shared → VPS → Cloud → Serverless

Latihan

1. Trace URL

Uraikan URL ini bagian per bagian:

`https://shop.example.com:8080/products/category?sort=price&limit=10#reviews`

2. Analisis Network Tab

Buka 3 website favorit. Catat: - Total request yang keluar - Request paling lambat - Tipe file paling banyak (gambar? JS? CSS?) - Status code 200 vs 404 vs 301

3. Gambar Arsitektur

Bikin diagram Mermaid arsitektur web yang mencakup: Client, DNS, CDN, Web Server, Database. Jelaskan alur request dari user buka browser sampai dapat halaman.

4. Bedain Hosting Types

Buat tabel perbandingan Shared, VPS, Cloud, Serverless dari segi: - Harga - Kontrol - Skalabilitas - Contoh kasus penggunaan - Kelebihan & kekurangan

02. HTTP Basics

Apa itu HTTP?

HTTP (HyperText Transfer Protocol) = bahasa yang dipake client & server buat komunikasi. Setiap interaksi web adalah pertukaran **HTTP Request** (dari client) dan **HTTP Response** (dari server).

Cara Kerja

sequenceDiagram

participant C as Client

participant S as Server

C->>S: 1. HTTP Request
GET /products HTTP/1.1
Host: api.example.com<

Note over S: 2. Server proses

S-->>C: 3. HTTP Response
HTTP/1.1 200 OK
Content-Type: application/j

C->>S: 4. Request lain
POST /products

S-->>C: 5. Response
201 Created

HTTP Methods

Method = *verb* yang ngasih tau server **mau ngapain**.

Method	Fungsi	Analogi Restoran	Kapan Pake
GET	Ambil data	Lihat menu	Baca artikel, liat produk
POST	Kirim data baru	Pesan makanan	Register, upload foto
PUT	Update data (ganti semua)	Ganti pesanan total	Update profil, ganti password
PATCH	Update data (sebagian)	Ganti topping aja	Edit bio, ubah quantity
DELETE	Hapus data	Batalan pesanan	Hapus akun, delete post

Catatan: PUT kirim data lengkap, PATCH kirim data yang berubah aja.

Contoh pake curl

curl https://api.github.com/users/midory

GET

curl -X POST https://api.example.com/users

POST

curl -X PUT -d '{"name":"Baru"}' https://api.example.com/users/1

PUT

curl -X DELETE https://api.example.com/users/1

DELETE

Idempotent & Safe

Sifat	Arti	Method
Safe	Gak ngubah data di server	GET
Idempotent	Request berkali-kali hasilnya sama	GET, PUT, DELETE
Not Safe	Bisa ngubah data	POST, PUT, PATCH, DELETE

HTTP Status Codes

Server ngasih kode 3 digit biar client tau hasilnya.

1xx — Informational

Lagi proses, tunggu bentar.

2xx — Success

Kode	Nama	Arti
200	OK	Sukses! Data dikirim
201	Created	Data baru berhasil dibuat
204	No Content	Sukses, tapi gak ada data (biasanya setelah DELETE)

3xx — Redirection

Kode	Nama	Arti
301	Moved Permanently	Pindah alamat permanen (cache oleh browser)
302	Found	Pindah sementara
304	Not Modified	Data gak berubah, pake cache aja

4xx — Client Error

Kode	Nama	Arti	Analogi
400	Bad Request	Request salah format	Pesan gak jelas
401	Unauthorized	Belum login	Kartu identitas gak ada
403	Forbidden	Gak punya akses	Dilarang masuk
404	Not Found	Resource gak ada	Alamat salah
405	Method Not Allowed	Method gak didukung	Minta bayar cash di toko cashless
429	Too Many Requests	Kebanyakan request, kena rate limit	Antrian penuh

5xx — Server Error

Kode	Nama	Arti
500	Internal Server Error	Error umum di server
502	Bad Gateway	Server gateway dapet response invalid
503	Service Unavailable	Server sibuk / maintenance

```
graph TD
    subgraph "Status Codes Overview"
        A[200 OK] -->|Success| B[Client request OK]
        C[404 Not Found] -->|Client Error| D[Resource missing]
        E[500 Error] -->|Server Error| F[Something broke]
    end
    style A fill:#c8e6c9
    style C fill:#ffccbc
    style E fill:#ffcdd2
```

HTTP Headers

Header = metadata yang nyertain request/response.

Request Headers

Header	Contoh	Fungsi
Host	api.example.com	Domain tujuan
User-Agent	Mozilla/5.0...	Identitas browser/client
Accept	application/json	Format response yang diinginkan
Authorization	Bearer eyJhbG...	Token autentikasi
Content-Type	application/json	Format data yang dikirim (POST/PUT)
Cookie	session_id=abc	Data session

Response Headers

Header	Contoh	Fungsi
Content-Type	text/html; charset=utf-8	Format response
Content-Length	1234	Ukuran response (bytes)
Set-Cookie	session_id=xyz; Path=/	Set cookie di browser
Cache-Control	max-age=3600	Aturan caching
Access-Control-Allow-Origin	*	CORS policy

```
# Liat headers pake curl -v
curl -v https://api.github.com
```

HTTPS & SSL

HTTP vs HTTPS

HTTP	HTTPS
Data plaintext	Data terenkripsi
Port 80	Port 443
Rentan man-in-the-middle	Aman pake TLS/SSL
❑ Trusted	❑ Padlock hijau di browser

Cara Kerja HTTPS

1. Client minta koneksi aman (ClientHello)
2. Server kirim sertifikat SSL + public key
3. Client verifikasi sertifikat (via Certificate Authority)
4. Client buat session key, enkrip pake public key

5. Server dekrip pake private key
6. Komunikasi terenkripsi dimulai!

SSL Certificate: Dapet gratis dari Let's Encrypt, atau bayar dari DigiCert, Cloudflare, dll.

HTTP/2 vs HTTP/3

Fitur	HTTP/1.1	HTTP/2	HTTP/3
Tahun	1997	2015	2022
Transport	TCP	TCP	QUIC (UDP)
Koneksi	1 request per koneksi	Multiplexing	Multiplexing
Head-of-line blocking	☐ Ya	△ Parsial (TCP level)	☐ Gak ada
Server push	☐	☐	☐
Adopsi	Legacy	40%+ web	30%+ (makin naik)

Multiplexing = kirim banyak request dalam 1 koneksi tanpa nunggu antrian.

Rangkuman

Konsep	Inti
HTTP Methods	GET ambil, POST buat, PUT ganti, PATCH edit, DELETE hapus
Status Codes	2xx ok, 3xx pindah, 4xx error client, 5xx error server
Headers	Metadata request/response
HTTPS	HTTP + enkripsi TLS, pake sertifikat SSL
HTTP/2 & /3	Lebih cepet, multiplexing, HOL blocking solved

Latihan

1. Match Method to Action

Cocokin method HTTP ke aksi berikut: - [] Ngirim form login - []
Liat daftar produk - [] Hapus komentar - [] Ganti nama profil - []
Upload foto profil

Tulis method yang tepat (GET, POST, PUT, PATCH, DELETE).

2. Identify Status Codes

Kasus berikut, status code apa yang tepat? - User buka halaman yang
gak ada - Form login berhasil - User gak login nyoba akses dashboard
- Server lagi down maintenance - Data baru berhasil dibuat - Redirect
dari http ke https

3. Curl Command Practice

Tulis curl command buat: - GET data user dari `https://jsonplaceholder.typicode.com/users/`
- POST user baru dengan data `{"name": "Budi", "email": "budi@mail.com"}`
- Liat response headers aja dari Google - DELETE post dengan id 1

4. HTTP vs HTTPS

Jelaskan pake diagram atau tulisan: - Apa bedanya HTTP dan HTTPS? -
Gimana proses handshake TLS? - Kenapa HTTPS penting buat produc-
tion? - Cari tau: website mana aja yang masih pake HTTP? Kenapa?

03. Frontend vs Backend

Gambaran Besar

```
graph LR
    subgraph "Frontend"
        A[HTML] -->|Struktur| D[Browser]
        B[CSS] -->|Styling| D
        C[JavaScript] -->|Interaksi| D
    end
    subgraph "API"
        E[REST API] --- F[JSON]
    end
    subgraph "Backend"
        G[Server Logic] --> H[(Database)]
        I[Auth] --> G
    end
```

```

end
D <--> E <--> G
style A fill:#e3f2fd
style B fill:#fff3e0
style C fill:#fff9c4
style E fill:#e1bee7
style H fill:#f3e5f5

```

Layer	Tugas	Teknologi
Frontend	Tampilan, interaksi user	HTML, CSS, JavaScript, React, Vue
API Backend	Jembatan FE ↔ BE Logic, data, auth, API	REST, JSON, GraphQL Node.js, Python, Go, database

Frontend

HTML — Struktur

```

<div class="product-card">
  <h2>MacBook Pro</h2>
  <p class="price">Rp 25.000.000</p>
  <button onclick="addToCart()">Beli</button>
</div>

```

CSS — Tampilan

```

.product-card {
  border: 1px solid #ddd;
  padding: 16px;
  border-radius: 8px;
}
.price { color: #e65100; font-weight: bold; }

```

JavaScript — Interaksi

```

function addToCart() {
  alert('Barang ditambahkan ke keranjang!');
  updateCartCount();
}

```

Cara Kerja Browser Rendering

HTML → DOM (Document Object Model)
CSS → CSSOM (CSS Object Model)
JS → Eksekusi + DOM manipulation

↓

Render Tree → Layout → Paint → Composite

Browser baca HTML → parse → bikin DOM tree → CSSOM →
render tree → gambar di layar.

Backend

Server Logic

```
// Contoh server sederhana pake Node.js (tanpa framework)
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/api/users' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([
      { id: 1, name: 'Budi', email: 'budi@mail.com' }
    ]));
  } else {
    res.writeHead(404);
    res.end('Not Found');
  }
});

server.listen(3000);
```

```
graph TD
  A[HTTP Request] --> B{Route Match?}
  B -->|Ya| C[Controller Logic]
  B -->|Tidak| D[404 Not Found]
  C --> E[Query DB]
  E --> F[Format Response]
  F --> G[Send JSON/HTML]
  D --> G
  style A fill:#bbdefb
  style G fill:#c8e6c9
```

Database

Database tempat nyimpen data aplikasi:

Jenis	Contoh	Cocok
Relational (SQL)	PostgreSQL, MySQL	Data terstruktur (user, order)
NoSQL	MongoDB, Firestore	Data fleksibel (JSON)
Cache	Redis	Data sementara (session)
Full-text Search	Elasticsearch	Pencarian

API — Jembatan FE & BE

API (Application Programming Interface) = cara dua aplikasi ngobrol.

sequenceDiagram

```
participant FE as Frontend (Browser)
participant API as REST API
participant BE as Backend
participant DB as Database
```

```
FE->>API: GET /api/products
API->>BE: Panggil handler
BE->>DB: SELECT * FROM products
DB-->>BE: Data products
BE-->>API: JSON response
API-->>FE: [{"id":1,"name":"Laptop",...}]
FE->>FE: Render ke HTML
```

REST API Concept

REST = Representational State Transfer. Prinsipnya:

Prinsip	Arti
Stateless	Tiap request berdiri sendiri, gak butuh konteks sebelumnya
Resource-based	Setiap entity punya endpoint: /users, /products, /orders

Prinsip	Arti
CRUD via HTTP	GET (read), POST (create), PUT/PATCH (update), DELETE
JSON Response	Data dikirim dalam format JSON

Contoh Endpoint REST

Method	Endpoint	Fungsi
GET	/api/users	Ambil semua user
GET	/api/users/1	Ambil user id 1
POST	/api/users	Buat user baru
PUT	/api/users/1	Update user id 1
DELETE	/api/users/1	Hapus user id 1

JSON Format

Contoh JSON

```
{
  "id": 1,
  "name": "Budi Santoso",
  "email": "budi@mail.com",
  "isActive": true,
  "hobbies": ["coding", "gaming"],
  "address": {
    "city": "Jakarta",
    "country": "Indonesia"
  }
}
```

JSON vs XML

JSON	XML
Ringan, lebih kecil	Berat, lebih besar
Gampang dibaca JS	Perlu parser
{ "key": "value" }	<key>value</key>
Array langsung []	Butuh <list><item>

API Testing — Postman / Thunder Client

Postman

1. Download Postman → New Request
2. Pilih method (GET, POST, dll)
3. Masukin URL: `https://jsonplaceholder.typicode.com/posts`
4. Klik **Send**
5. Liat response: status, body JSON, headers

Thunder Client (VS Code Extension)

Lebih ringan, langsung dari VS Code.

Test Public API

Coba endpoint ini pake Postman/Thunder Client:

Endpoint	Method	Coba
<code>https://jsonplaceholder.typicode.com/posts</code>	GET	Ambil semua posts
<code>https://jsonplaceholder.typicode.com/posts/1</code>	GET	Ambil 1 post
<code>https://jsonplaceholder.typicode.com/posts</code>	POST	Buat post baru (isi Body JSON)
<code>https://api.github.com/users/octocat</code>	GET	Ambil data user GitHub

```
// Body buat POST
{
  "title": "Belajar API",
  "body": "Seru banget!",
  "userId": 1
}
```

Serving Static Files vs Dynamic Content

Static	Dynamic
File udah jadi (.html, .css, .js) Langsung dikirim ke browser	Dibikin server pas request Proses dulu (query DB, logic)

Static	Dynamic
Cepet, bisa cache CDN	Lebih lambat, butuh server
Contoh: profile page, blog post	Contoh: dashboard user, search

```
graph LR
    subgraph "Static"
        A[HTML File] -->|Langsung| B[Browser]
    end
    subgraph "Dynamic"
        C[Request] --> D[Server Logic]
        D --> E[DB Query]
        E --> F[Generate HTML/JSON]
        F --> G[Browser]
    end
    style A fill:#c8e6c9
    style D fill:#fff9c4
    style F fill:#ffccbc
```

Rangkuman

Konsep	Inti
Frontend	HTML (struktur) + CSS (tampilan) + JS (interaksi), render di browser
Backend	Logic, database, API, jalan di server
API	Jembatan FE-BE via HTTP, biasanya REST + JSON
REST	Resource-based endpoints, CRUD via HTTP methods
JSON	Format data ringan pengganti XML
Static vs Dynamic	File siap vs content generated on request

Latihan

1. Bedain FE vs BE

Kategorikan ini Frontend (FE) atau Backend (BE): - [] Nampilin data user di halaman profil - [] Validasi password sebelum simpan ke DB - [] Ngririm email notifikasi - [] Animasi tombol hover - [] Routing URL di browser (React Router) - [] Query database buat laporan - [] Format response JSON dari API

2. Parse JSON

Dari JSON response API berikut, jawab:

```
{
  "status": "success",
  "data": {
    "users": [
      { "id": 1, "name": "Budi", "email": "budi@mail.com", "role": "admin" },
      { "id": 2, "name": "Sari", "email": "sari@mail.com", "role": "user" },
      { "id": 3, "name": "Joko", "email": "joko@mail.com", "role": "user" }
    ],
    "total": 3,
    "page": 1
  }
}
```

- Ada berapa user?
- Siapa aja yang role admin?
- Apa status dari response ini?
- Gimana cara akses email user pertama di JavaScript?

3. API Test

Pake Postman/Thunder Client/curl, lakukan: - GET posts dari <https://jsonplaceholder.typicode.com/posts> - GET post id 42 - POST post baru (data bebas) - Catat: status code tiap response, response time, content-type header

4. Gambar Arsitektur Fullstack

Bikin diagram Mermaid yang mencakup: - User dengan browser - Frontend (React/HTML) - REST API - Backend server - Database (PostgreSQL) - Alur data dari user login sampai liat profil

04. Hosting & Deployment

Hosting Flow

```
graph TD
  A[Punya Code] --> B{Pilih Hosting}
  B -->|Frontend| C[Vercel / Netlify / GitHub Pages]
  B -->|Backend| D[Railway / Render / Fly.io]
  B -->|Database| E[Neon / Supabase / PlanetScale]
  C --> F[Push ke Git]
  D --> F
  E --> F
  F --> G[Auto Deploy!]
  G --> H[Live di Internet]
  style A fill:#e3f2fd
  style C fill:#c8e6c9
  style D fill:#fff9c4
  style E fill:#e1bee7
  style H fill:#ffccbc
```

Static Hosting

Vercel

- Cocok: Next.js, React, Vue, static HTML
- Fitur: Auto deploy dari GitHub, CDN global, custom domain, serverless functions
- Harga: Free tier generous (100GB bandwidth/bln)

Netlify

- Cocok: Static site, Jamstack, Hugo, Gatsby
- Fitur: Forms, functions, split testing, deploy previews
- Harga: Free tier (100GB bandwidth/bln)

GitHub Pages

- Cocok: Dokumentasi, portfolio, project pages
- Fitur: Gratis, auto deploy dari GitHub Actions
- Batasan: Gak bisa backend/server-side

Note: Semua static hosting support custom domain + HTTPS otomatis (pake Let's Encrypt).

Deploy Static Site — Langkah Praktis

Via Vercel:

```
# 1. Install Vercel CLI  
npm i -g vercel
```

```
# 2. Deploy dari folder  
vercel --prod
```

```
# 3. Ikutin wizard — selesai!
```

Via GitHub Pages: 1. Bikin repo GitHub 2. Push HTML/CSS ke main branch 3. Settings → Pages → pilih main → Save 4. Selesai!
<https://username.github.io/repo/>

Backend Hosting

Railway

- Cocok: Node.js, Python, Go, Docker
- Fitur: Auto deploy dari GitHub, environment variables, database add-on
- Harga: Free tier 500 jam/bln

Render

- Cocok: Web services, cron jobs, static sites
- Fitur: Auto HTTPS, custom domain, persistent disk
- Harga: Free tier (slow spin-up — cold start)

Fly.io

- Cocok: Docker apps, edge deployment
- Fitur: Deploy dekat user (edge), IPv6, global
- Harga: Free tier 3 VMs

```
// Contoh deploy minimal — Express.js buat Railway/Render  
const express = require('express');  
const app = express();  
const PORT = process.env.PORT || 3000;
```

```
app.get('/', (req, res) => {  
  res.send('Hello from Railway!');  
});
```

```
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Database Hosting

Neon (PostgreSQL)

- Fitur: Serverless Postgres, branching, auto-scale
- Harga: Free tier 500MB storage
- Cocok: Aplikasi modern, serverless

Supabase (PostgreSQL + Backend)

- Fitur: Database + Auth + Storage + Realtime
- Harga: Free tier 500MB
- Cocok: Full-stack app pengganti Firebase

PlanetScale (MySQL-compatible)

- Fitur: Branching database (kayak git!), auto-scale
 - Harga: Free tier 1GB storage
 - Cocok: Aplikasi MySQL dengan branching workflow
-

Custom Domain & DNS

Cara Kerja DNS

```
graph LR  
  A[User] -->|ketik: mywebsite.com| B[DNS Resolver]  
  B -->|tanya: dimana mywebsite.com?| C[DNS Server]  
  C -->|jawab: IP 192.0.2.1| B  
  B -->|arahkan ke| D[Hosting Server]  
  D -->|kirim HTML| A  
  style A fill:#bbdefb  
  style C fill:#fff9c4  
  style D fill:#c8e6c9
```

DNS Records

Record	Fungsi	Contoh
A	Domain → IPv4	@ → 76.76.21.21
AAAA	Domain → IPv6	@ → 2600:...
CNAME	Domain → Domain lain	www → myapp.vercel.app
MX	Domain → Mail server	@ → mail.google.com
TXT	Teks verifikasi	SPF, DKIM, domain verification

Setup Custom Domain

1. Beli domain dari registrar (Niagahoster, Namecheap, Cloudflare)
2. Arahkan DNS ke hosting provider
3. Di hosting (Vercel/Netlify): add custom domain
4. Tunggu propagasi DNS (5 menit — 24 jam)
5. HTTPS otomatis aktif

Cek propagasi DNS

```
dig mywebsite.com +short
nslookup mywebsite.com
```

Environment Variables

Environment variable (env vars) = konfigurasi rahasia yang gak ikut ke code.

Variable	Contoh	Kenapa Rahasia
DATABASE_URL	postgres://user:pass@host:5432/db	Kredensial DB
API_KEY	sk-abc123...	Token API eksternal
JWT_SECRET	supersecret123	Signature token
PORT	3000	Port server (bukan rahasia)

Cara Pake

```
# File: .env (jangan di-commit!)
DATABASE_URL=postgres://user:pass@neon.tech/db
API_KEY=sk-test123
PORT=3000
```

```
// Di code
require('dotenv').config();
```

```
const db = process.env.DATABASE_URL;
const port = process.env.PORT || 3000;
```

⚠ **Jangan commit .env!** Tambahin ke .gitignore.

Di hosting (Railway/Render/Vercel), set env vars lewat dashboard → Settings → Environment Variables.

Deployment Workflow (CI/CD)

CI/CD = Continuous Integration / Continuous Deployment

```
graph LR
  A[Push Code] -->|GitHub| B[CI Pipeline]
  B -->|Tes otomatis| C{Tests Pass?}
  C -->|Ya| D[Build]
  D --> E[Deploy ke Production]
  C -->|Tidak| F[Fix Bug]
  F --> A
  style A fill:#e3f2fd
  style B fill:#fff3e0
  style D fill:#c8e6c9
  style E fill:#a5d6a7
  style F fill:#ffcdd2
```

GitHub Actions Contoh

```
# .github/workflows/deploy.yml
name: Deploy to Vercel

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm run build
      - uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.ORG_ID }
          vercel-project-id: ${ secrets.PROJECT_ID }
```

Basic Deployment Checklist

Step	Detail
1	Code ready, tested locally
2	Push ke GitHub
3	CI run tests (lint, build, unit test)
4	Build production
5	Set environment variables di dashboard
6	Deploy (auto/ manual)
7	Test production URL
8	Setup custom domain
9	Enable HTTPS
10	Monitor (error, traffic, uptime)

Rangkuman

Konsep	Inti
Static Hosting	Vercel/Netlify/GitHub Pages — deploy dari Git, auto HTTPS
Backend Hosting	Railway/Render/Fly — jalanin server code
DB Hosting	Neon/Supabase/PlanetScale — database managed
DNS	Domain → IP, pake A/CNAME/MX records
Env Vars	Konfigurasi rahasia di .env, gak ikut code
CI/CD	Push → Test → Build → Deploy otomatis

Latihan

1. Deploy Static Site

Buat halaman portfolio sederhana (HTML + CSS) dan deploy ke **salah satu**: - Vercel (paling gampang) - GitHub Pages - Netlify

Hasil: URL live bisa diakses publik.

Template minimal:

```
<!DOCTYPE html>
<html>
<head>
  <title>Portfolio</title>
  <style>
    body { font-family: sans-serif; max-width: 600px; margin: auto; }
  </style>
</head>
<body>
  <h1>Halo, saya [Nama]</h1>
  <p>Ini portfolio pertama saya yang di-host di internet!</p>
</body>
</html>
```

2. Setup Custom Domain

(Bonus — klo punya domain) - Beli domain murah (Rp 15-20rb di Ni-agahoster) - Arahkan DNS ke Vercel/Netlify - Screenshot hasil DNS records + HTTPS active

3. Environment Variables

Buat file .env untuk aplikasi todo sederhana. Isi dengan: - DATABASE_URL — URL koneksi database (fake) - JWT_SECRET — secret key (fake) - PORT — port server - NODE_ENV — environment mode

Jelaskan kenapa env vars gak boleh di-commit ke GitHub.

4. CI/CD Diagram

Bikin diagram Mermaid deployment pipeline untuk aplikasi fullstack: - Developer push code ke GitHub - GitHub Actions run tests - Build frontend + backend - Deploy frontend ke Vercel - Deploy backend ke Railway - Migrasi database di Neon - Notifikasi ke Slack/Email

Sertain penjelasan tiap tahap.